

Labo 05 — Multi-stage et images de qualité production

Théorie — comment passer d'une image de 500 Mo qui contient votre compilateur à une image de 200 Mo qui ne contient que ce qui tourne ; et ce que Buildah fait à la place de BuildKit.

Objectifs

- Comprendre pourquoi une image qui sert à **construire** ne doit pas être celle qui **exécute**.
- Écrire un build **multi-stage** pour Spring Boot et pour Angular.
- Choisir une image de base en connaissance de cause (Debian/Ubuntu, Alpine, distroless).
- Savoir ce que BuildKit (Docker) et Buildah (Podman) apportent : cache, secrets, stages.
- Relier taille d'image, surface d'attaque et temps de déploiement.

1. Le problème : l'outillage de build reste dans l'image

Un premier Dockerfile naïf pour une API Java :

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package -DskipTests
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

Il fonctionne. Il produit une image de **800 Mo à 1 Go** qui contient : un JDK complet (compilateur, outils de débogage), Maven, le dépôt local `~/.m2` avec des centaines de JAR, votre **code source**, les tests, et le JAR final. En production, il ne sert que le JRE et le JAR : environ 200 Mo.

Les conséquences ne sont pas seulement esthétiques :

- **Sécurité.** Le code source part chez tous ceux qui ont l'image. Chaque outil embarqué (compilateur, `curl`, `git`, `shell`) est un moyen d'action supplémentaire pour un attaquant, et une ligne de plus dans le rapport de scan de vulnérabilités.
- **Coût.** 800 Mo transférés à chaque déploiement, sur chaque nœud, stockés dans chaque registry, avec la rétention des versions antérieures.
- **Temps.** Le démarrage d'un conteneur inclut le téléchargement de l'image si elle est absente. Sur un incident de production à 3 h du matin, la différence se voit.

→ SÉCURITÉ

La **surface d'attaque** d'une image, c'est l'ensemble de ce qu'un attaquant peut *utiliser* une fois qu'il a obtenu l'exécution de code : un shell pour explorer, `curl` pour exfiltrer, un compilateur pour fabriquer un outil, un gestionnaire de paquets pour en installer d'autres. Chaque binaire absent est une étape de plus pour lui. C'est pourquoi les scanners (Trivy, Gype) comptent les paquets, et pourquoi « minimal » n'est pas qu'une affaire de mégaoctets.

2. Le multi-stage

Un Dockerfile peut contenir **plusieurs FROM**. Chacun ouvre un *stage* — un environnement de construction indépendant. Seul le **dernier** produit l'image finale ; les autres sont jetés. Et `COPY --from=<stage>` permet de récupérer des fichiers dans un stage précédent.

```
# ----- stage 1 : construction -----
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ----- stage 2 : exécution -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/api.jar app.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

L'image finale contient **un JRE et un JAR**. Ni Maven, ni JDK, ni sources, ni tests, ni `.m2`. Rien de ce qui s'est passé dans le stage `build` n'y laisse de trace — pas même dans les couches masquées, puisque ces couches ne font tout simplement pas partie de l'image.

À RETENIR

Le multi-stage est aussi la seule protection réellement fiable contre les secrets de build : ce qui est copié dans un stage jeté n'existe pas dans l'image finale. Attention toutefois : `COPY --from=build /app /app` recopierait tout, secrets compris. On ne copie que l'artefact.

Le même schéma pour Angular :

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # produit dist/

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist/mon-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

→ ANGULAR

`ng build` compile les composants TypeScript, regroupe le tout en quelques fichiers `.js`, `.css` et un `index.html`, les minifie et les nomme avec une empreinte (`main-a1b2c3.js`) pour le cache navigateur. Le résultat est **statique** : n'importe quel serveur de fichiers le sert. `ng serve`, lui, est un serveur de développement qui recompile à chaque modification — précieux sur votre poste, sans objet en production.

Point crucial : **Node ne survit pas** au build. Un front Angular en production n'est que du HTML, du CSS et du JavaScript ; il n'a pas besoin d'un moteur JavaScript côté serveur. On ne conteneurise **jamais** `ng serve`.

3. Choisir son image de base

Base	Taille	Avantages	Inconvénients
<code>debian</code> / <code>ubuntu</code>	75-120 Mo	Tout fonctionne, outillage complet, <code>glibc</code>	Lourde, beaucoup de paquets donc de CVE
<code>*-slim</code>	30-80 Mo	Bon compromis, reste du Debian	Moins d'outils installés
<code>alpine</code>	5-10 Mo	Très légère, <code>apk</code> efficace	Utilise musl et non <code>glibc</code>
<code>distroless</code>	20-50 Mo	Aucun shell, aucun gestionnaire de paquets	Débogage difficile, pas de <code>exec sh</code>

→ LINUX

La **bibliothèque C** (`libc`) est la couche entre les programmes et le noyau : `printf` , `malloc` , la résolution DNS, les locales. Presque tout binaire Linux en dépend. `glibc` (GNU) est l'implémentation historique, riche et compatible ; `musl` est une réécriture minimaliste, choisie par Alpine pour sa taille. Un binaire compilé pour l'une ne se charge pas avec l'autre : `ldd --version` dans le conteneur vous dit laquelle vous avez.

Le piège d'Alpine mérite un développement. La plupart des programmes s'accommodent de `musl` , mais pas tous : les binaires natifs compilés pour `glibc` refusent de démarrer, certaines bibliothèques Java natives (compression, cryptographie, génération de PDF) échouent avec `UnsatisfiedLinkError` , et des différences de résolution DNS ou de locales apparaissent. Des ralentissements ont aussi été mesurés sur l'allocation mémoire de certaines charges Java.

En pratique, pour Spring Boot : `eclipse-temurin:21-jre-alpine` convient dans l'immense majorité des cas et divise la taille par deux ; en cas de dépendance native, on revient à `eclipse-temurin:21-jre` (Ubuntu). Le choix se **teste**, il ne se décrète pas.

Les images `distroless` (Google) ne contiennent que le runtime et votre application : pas de shell, pas de `ls` , pas de gestionnaire de paquets. La surface d'attaque est minimale, mais `podman exec -it conteneur sh` ne fonctionne plus — il faut avoir prévu son observabilité autrement.

4. Ce qui pèse vraiment

Quatre leviers, par ordre d'efficacité décroissante :

1. **Le multi-stage** — supprime l'outillage de build. C'est le levier majeur : 800 Mo → 200 Mo.
2. **L'image de base** — `-jre` au lieu de `-jdk` , `alpine` au lieu d' `ubuntu` .
3. **Le `.dockerignore`** — évite d'embarquer `.git` , `node_modules` , `target` .
4. **Le regroupement installation/nettoyage** dans le même `RUN` .

En revanche, ce qui n'a **aucun** effet : supprimer des fichiers dans une couche ultérieure. Ils restent dans l'image (labo 02). Et le nombre de couches, à lui seul, ne change presque rien à la taille.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman history mon-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
```

5. BuildKit et Buildah

Docker construit avec **BuildKit** ; Podman construit avec **Buildah**. Les deux lisent le même Dockerfile et offrent les mêmes fonctions utiles :

- **Les stages inutiles ne sont pas construits.** Un stage dont rien n'est copié dans l'image finale est ignoré — Buildah affiche `[2/3]` , `[3/3]` et saute le `[1/3]` .
- `--target` construit jusqu'à un stage donné : `podman build --target build -t api-build .` vous donne l'image du stage de compilation, pour l'inspecter.
- **Les caches persistants.** `RUN --mount=type=cache,target=/root/.m2 mvn package` conserve le dépôt Maven **entre les builds**, sans l'inclure dans l'image. Sur un agent de CI, le gain est spectaculaire.
- **Les secrets.** `RUN --mount=type=secret,id=npnrc ...` rend un fichier disponible pendant une seule instruction, sans jamais l'écrire dans une couche (labo 08).

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests
```

PODMAN

Une différence visible : BuildKit construit les stages indépendants **en parallèle**, Buildah les construit l'un après l'autre. Une autre : la ligne `# syntax=docker/dockerfile:1` , qui active la syntaxe étendue chez Docker, est simplement **ignorée** par Buildah — les `--mount` fonctionnent sans elle. Et le cache de `type=cache` vit dans votre stockage utilisateur (`~/.local/share/containers/storage`), pas dans un daemon : deux utilisateurs du même serveur de CI ne le partagent pas.

6. Spring Boot : les couches du JAR

Un JAR Spring Boot pèse 50 Mo, dont 45 Mo de dépendances qui ne changent presque jamais et 5 Mo de code qui change à chaque commit. Copié en bloc, il forme une couche unique de 50 Mo retransférée intégralement à chaque déploiement. Spring Boot sait se découper :

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY target/api.jar api.jar
RUN java -Djarmode=tools -jar api.jar extract --layers --destination extracted

FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/extracted/dependencies/ ./
COPY --from=extract /app/extracted/spring-boot-loader/ ./
COPY --from=extract /app/extracted/snapshot-dependencies/ ./
COPY --from=extract /app/extracted/application/ ./
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Les dépendances forment une couche stable, le code une petite couche volatile : le déploiement ne transfère plus que quelques Mo. Retenez le **principe**, qui est celui du labo 04 appliqué au contenu d'un JAR.

7. En entreprise

- **Un seul Dockerfile par service**, multi-stage, versionné avec le code. La CI n'a besoin ni de Maven ni de Node : `podman build` (ou `docker build`) suffit, ce qui garantit que le build de la CI et celui du poste sont identiques.
 - **Les tests** tournent souvent dans un stage dédié (`RUN mvn test`), pour qu'un test rouge fasse échouer le build de l'image.
 - **Le scan de vulnérabilités** (Trivy, Gype) s'applique à l'image finale. Une image minimale produit un rapport court, donc réellement traité — une image de 1 Go produit 300 CVE que personne ne lira.
 - **L'image finale tourne en non-root**, sur un port > 1024, sans shell si possible.
-

À retenir

- Ce qui construit ne doit pas exécuter : c'est tout l'objet du multi-stage.
- Plusieurs `FROM` = plusieurs stages ; seul le dernier devient l'image, `COPY --from` y récupère l'artefact.
- Node n'a rien à faire dans l'image finale d'un front Angular : on sert du statique avec nginx.
- `-jre` plutôt que `-jdk`, `alpine` si les dépendances natives le permettent, `distroless` si l'on accepte de perdre le shell.
- Alpine utilise `musl` et non `glibc` : à valider par un test, jamais par principe.
- BuildKit et Buildah offrent `--target`, les caches persistants et les secrets de build ; Buildah ignore `# syntax=` et ne parallélise pas.
- Supprimer un fichier dans une couche ultérieure ne réduit pas la taille de l'image.

Vocabulaire

stage : étape de build ouverte par un `FROM`. — `COPY --from` : récupération de fichiers depuis un autre stage ou une autre image. — `--target` : arrêter le build à un stage donné. — **distroless** : image sans shell ni gestionnaire de paquets. — **musl / glibc** : deux implémentations de la bibliothèque C. — **BuildKit / Buildah** : moteurs de build de Docker et de Podman. — **cache mount** : cache persistant entre builds, hors image. — **surface d'attaque** : ensemble des composants exploitables présents dans l'image.